



Tencent fires its editorial staff

Tencent has laid off all of the editorial staff at Fanbyte, an online gaming publication owned by it. <https://dgit.in/oct22-57>



Less power hungry. Irony.

Apple is reportedly researching how it can save power by automatically pausing media playback. <https://dgit.in/oct22-58>

memory throughput. The throughput can vary vastly depending on the access patterns for each type of memory so organising the memory accesses as per the device specifica-

width. Bandwidth that can be used for other transfers. And if the system comes with a FSB or a Front-Side Bus, then you can use Page-Locked Host Memory to achieve higher

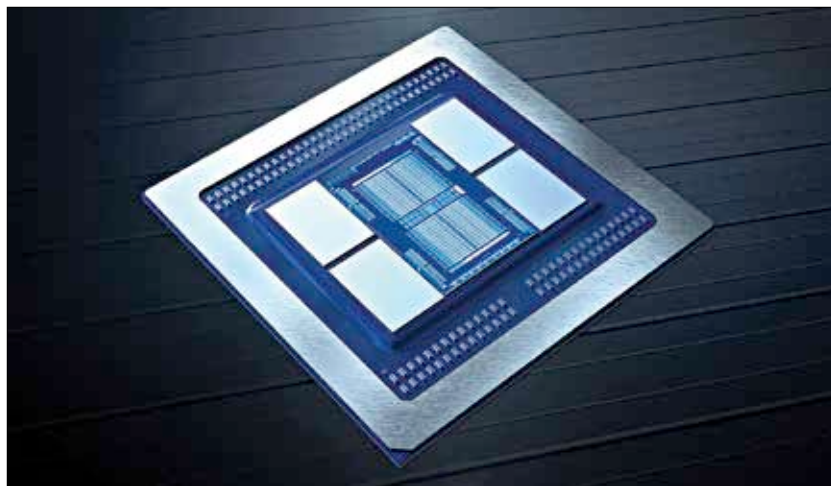
worthless and mapped Page-Locked memory should be used.

MAXIMISE INSTRUCTION THROUGHPUT

Maximising instruction throughput requires a bit more know how regarding how different compilers treat the code that you write. Modern day compilers will encourage the use of certain primitives that are more efficient to begin with. With CUDA, there are a few simple strategies that you can follow.

Minimising the use of arithmetic instructions with low throughput. You might have to trade precision for speed by using single-precision functions rather than double-precision functions. Obviously, if you need double-precision then by all means you should use it. Otherwise, stick to single-precision. And then you should use intrinsic functions instead of regular functions. Intrinsic functions can only be used in device mode and they're often less accurate than standard functions. You can also use the compiler flag `-use_fast_math` at compile time to force the compiler to use intrinsic functions wherever you have used regular functions. Such blank approaches will drop accuracy by a significant amount. An easy way to identify intrinsic functions is by the double underscore that's prefixed before the function name. For example, `_tanf(x)` is the intrinsic version of `tanf(x)`. Also, flushing denormalised numbers to zero helps.

Reduce the number of divergent warps caused by control flow instructions. Control flow instructions such as `if`, `switch`, `for`, `while`, `do`, etc. can impact instruction throughput by a huge amount by causing threads of the same warp to follow different execution paths i.e. diverge. Whenever this happens, you lose out on parallelisation because there is a dependency being created which is waiting for another iteration or run to be completed. This makes the workload serialised for the warp. As mentioned previously,



HBM memory with up to 665 GB/s bandwidth

tions is optimal. This is very much applicable in places where global memory access is required frequently.

Host and device bandwidth

As mentioned previously, data transfer between the host and the device should be minimised because the bandwidth is low and that subsequently holds up execution as threads have to wait longer for data. One way to minimise this would be by moving more of the code to the device itself. This would mean that some kernels which do not have potential for parallelism are also executed on the device. While this might seem like a non-optimal way of going about doing things, the trade-off is actually not that bad. One of the ways this is resolved on the device is when Intermediate data structures are created and destroyed without ever being written to the host memory.

Data transfers have overheads and that can also chalk up to a lot of bandwidth getting used up if you make a lot of small transfers between the host and the device. Batching a lot of these small transfers together and then sending them in one go reduces the overheads and saves on band-

performance with transfers. Regular pageable host memory is allocated using `malloc()` but with CUDA, you can have Page-Locked Host Memory. You can allocate Page-Locked Host Memory using `cudaHostAlloc()` and `cudaFreeHost()` to allocate and then later free up the memory. Also, `cudaHostRegister()` page-locks a range of memory that has already been allocated using `malloc()`. The benefits of using Page-Locked Host Memory is that you can use kernel execution to concurrently work on copies stored between the Page-Locked Host Memory and the device memory. On certain devices, the Page-Locked Host Memory can be mapped onto the address space of the device which makes it much easier for the device to process it since there is no need to copy the data from or to the device memory. Lastly, on systems with an FSB, the bandwidth is higher if the host memory is allocated as Page-Locked and the bandwidth can be even higher if it is allocated as write-combining. Certain integrated systems might have the host and the device memory being the same. In such cases, copying data between the host and the device memory is